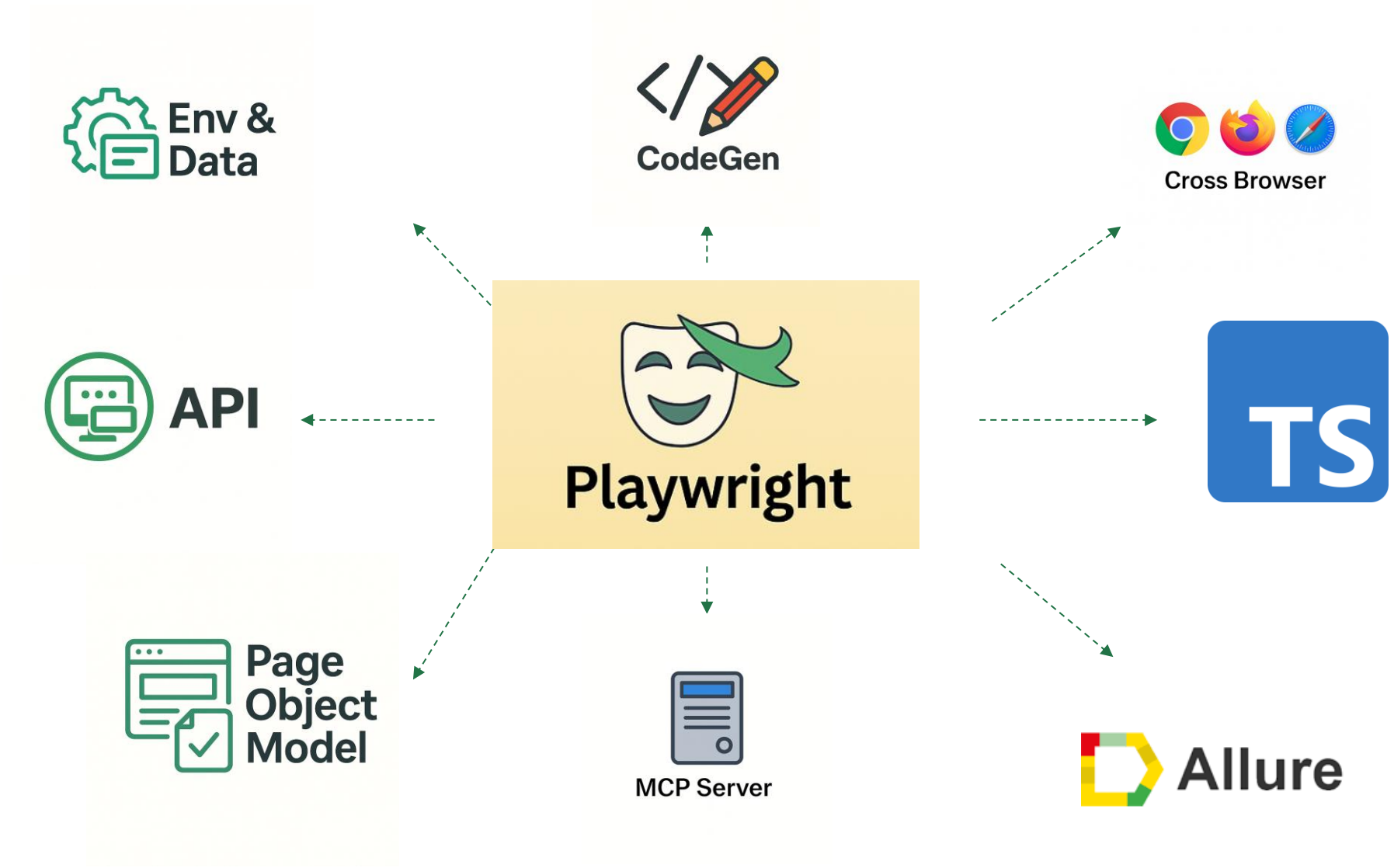
Playwright – E2E Automation Framework

Cope Automation

Playwright Framework Components

Cope

👋 Playwright – E2E Test Framework Components



Playwright Overview

Cope

🎯 What is Playwright?

"A **person** who writes plays."

Cope

🎯 Why Playwright?

Top 5 Reasons

1.3x cross: Platform, Browsers (And Devices), Language

2.Own Built-in test runner with inbuilt Assertions

3.Rich features like auto-waiting, code-gen, annotations, tags, many reporters

4.Integrated API Testing

5.Powerful MCP server - backed up Microsoft

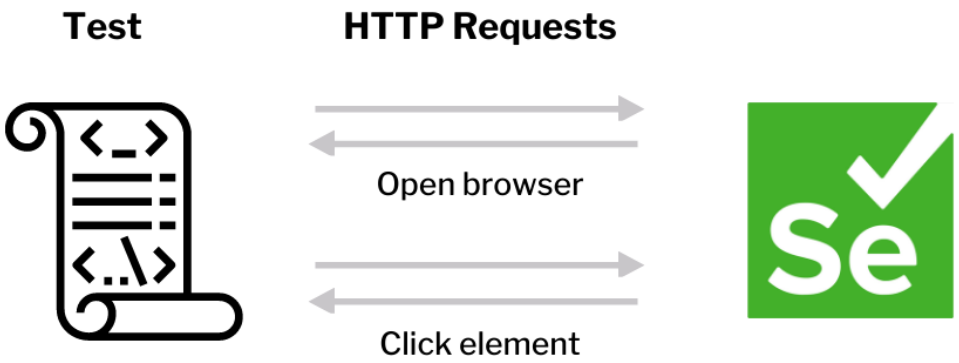
Cope

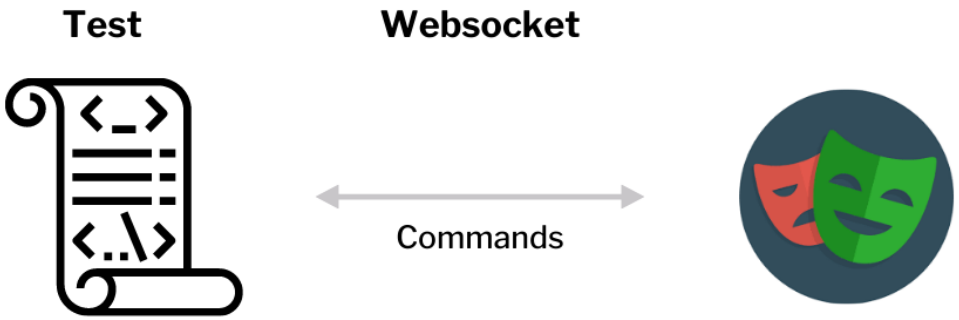
Playwright Architecture

Cope

🎯 Playwright Architecture

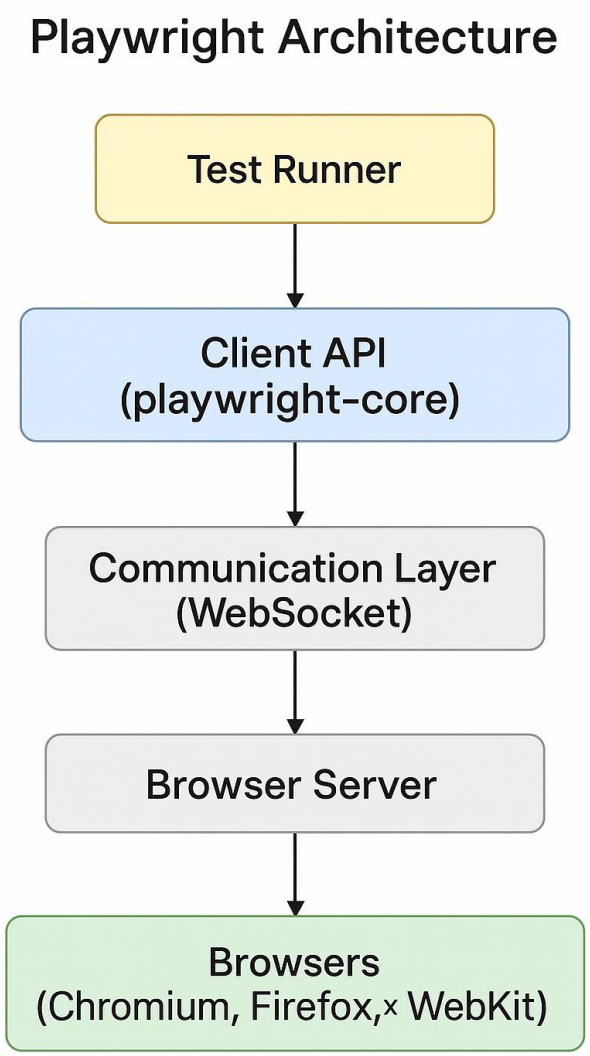
Playwright - the communication happens via secure **websocket** protocol





Cope

🎯 Playwright Test Runner



Format of the Course

🎯 What is my commitment to the course?

My commitments

1. Keep the content up-to-date as much as possible
2. Responding to your queries as quickly as I can
3. Providing additional resources like (.md file, code reference)

Cope

🎯 What do I expect from you?

What can you do for me?

1. Your time commitment to the learn and implement
2. Provide enough information when raising questions
3. Feedback

Cope

Thank you !

Let’s get started 🚀

Playwright Introduction

Cope

Installation

Cope

What is Node.js ?

It’s a ***runtime environment*** for running JavaScript code.

With a TypeScript compiler, it runs TypeScript code as well.

Cope

Set up first node project

1. Create a folder (preferably under `/documents/workspace/<name-of-the-project`)
2. Initialize node project
   - `npm init [-y]`
3. Check `package.json` is created
4. Run a simple JS code
5. Done ! 🎉

🎯 Best Practices - Naming Conventions

Naming Convention - Best Practice:

16.2. File & Code Naming Conventions

Item	Convention	Example
Folders / Files	kebab-case	`page-objects/`, `file-helper.ts`
Page and Spec files	dot-separated	`nopcommerce.home.page.ts`, `nopcommerce.e2e.spec.ts`
Class Names	PascalCase (each word capitalized)	`LoginPage`, `DashboardActions`
Variables	camelCase	`loginButton`, `userNameInput`
Constants	UPPER_SNAKE_CASE	`BASE_URL`, `API_TIMEOUT_MS`

🎯 **Consistent naming improves readability and reduces confusion across teams.**

Cope

GitHub Integration

Cope

Git - Overview

"Git is a distributed **version control** software/system, It is a free and open source project."

What is it used for?

1. Version controlling the code
2. Branching Strategy (feature, test, prod, master)
3. Team Collaborations
4. Tracking Change History
5. CI/CD Integration

Cope

First Playwright

Cope

💡 Did you know ?

What is Playwright?

" A person who writes plays."

Cope

📌 Rules About Playwright Spec/Test file...

Playwright recognizes the following file extensions as **valid test specification files**

- *.spec.ts
- *.test.ts

Cope

📌 Rules About Playwright Spec/Test file...

Playwright recognizes the following file extensions as **valid test specification files**

- *.spec.ts
- *.test.ts

Cope

Understanding Core Concepts

Cope

Playwright Test Runner

Cope

🎯 Key Notes on “Playwright Test Runner”

1.Own built-in Test Runner (No external runners like Mocha, Jest, Jasmine)

2. Powerful Config file (Controls overall test settings incl browsers, reporters, parallel runs)

3. **test**, **expect**, **request** some of the most used functions

MOCHA

Jest

AXIOS

Cope

🔥 Most Used Other Imports from Playwright Runner

```ts
export const chromium
export const firefox
export const webkit
export const selectors
export const devices
export const errors
export const request
export const _electron
export const _android
export const test
export const expect
export const defineConfig
export const mergeTests
export const mergeExpects
export default playwright.test;
```

Playwright Test Function

Cope

🎯 Signature of *test()* function

Declares a test.

- `test(title, body)`
- `test(title, details, body)`

Cope

🎯 Signature of *test()* function

Declares a test.

- `test(title, body)`
- `test(title, details, body)`

@param `title` — Test title.

@param `details` — Additional test details.

@param `body`

Test body that takes one or two arguments: an object with fixtures and optional TestInfo.

Cope

🎯 Signature of *test()* function

Declares a test.

Title as **string**

- `test(title, body)`
- `test(title, details, body)`

Cope

🎯 Signature of *test()* function

Body as callback function **{<fixture e.g. page>}, [testInfo]**

Declares a test.

- `test(title, body)`
- `test(title, details, body)`

Cope

🎯 Signature of *test()* function

Declares a test.

Details as **{<additional details like tag>}**

- `test(title, body)`
- `test(title, details, body)`

Cope

🔥 Most common Body args

```
[🔥] A test body take one of the following args
  - page
  - context
  - request
```

Cope

Page Fixtures

Cope

DRY Principle

DRY - Don’t Repeat Yourself

Cope

🎯 Page - Fixture

A page is a **fixture** from the playwright runner's perspective, which is passed as an arg to the test object.

✅ Most Commonly Used Methods

Method	Description
`goto(url)`	Navigate to a URL
`locator(selector)`	Create a locator for actions/assertions
`getByRole()`, `getByText()`, `getByLabel()`	Semantic locators for accessibility-based selectors
`click(selector)`	Clicks on a matching element
`fill(selector, value)`	Fills an input field
`evaluate(fn, arg)`	Runs JavaScript in the browser context
`waitForURL(url)`	Wait for navigation to a specific URL
`screenshot()`	Capture a screenshot
`title()`	Get the page title
`url()`	Get the current URL

** Note: The **'locator'** and **'getBy*'** methods return a **locator** objects

Cope

Locator Objects

Cope

🎯 Locator / Selector Object

Locators are the central piece of Playwright's auto-waiting and retry-ability. In a nutshell, locators represent a way to find element(s) on the page at any moment. A locator can be created with the page.locator() method.

✅ Most Commonly Used Methods

Method	Description
`click()`	Click on the located element
`fill(value)`	Fill an input or textarea with the specified value
`press(key)`	Press a keyboard key on the focused element
`getAttribute(name)`	Retrieve a specific attribute from the element
`textContent()`	Retrieve the raw text content of the element
`isVisible()`	Check if the element is visible
`isHidden()`	Check if the element is hidden
`isChecked()`	Check if the element is selected (for checkbox/radio)
`check()` / `uncheck()`	Set checkbox/radio to checked or unchecked
`selectOption()`	Select an option from a dropdown
`evaluate(fn)`	Execute JS in the page context for the matched element
`screenshot()`	Take a screenshot of the matched element

Cope

Expect Function

Cope

🎯 Expect Function

What it's signature?

```ts
expect(actual: any): Matchers<unknown>;
```

How to read the above signature?

The expect method takes any type of **'value'** (e.g. a page, locator, a number, storing, object, array) and then **returns a Matchers**.

Depends on the return type fo the matchers, the assertions methods can be used.

Cope

The “await” keyword explained!

Cope

🎯 Where to be awaited?

The await keyword is used in JavaScript and TypeScript to **pause the execution** of an async function until a Promise is resolved (success) or rejected (error). It helps you write asynchronous code that looks and behaves like synchronous code.

1. await before every **"page"** or **'locator'** action methods (e.g. click() and fill() ..etc)
2. await before **"expect"** methods

Cope

Codegen & lts usecases

Cope

🎯 Codegen - Overview

What it does?

1. Records test flow
2. Generates best locators

What are its benefits?

- ✅ Reduces test writing time drastically
- ✅ No more brittle/flaky selectors
- ✅ 'Record at cursor' - Enables adding locators in the middle of test flow

A big productivity Boost

Cope

Locator Strategies

Cope

🎯 Deep Dive into Playwright Locators

Key Takeaways

- ✅ page.getBy*() and page.locator() -> Returns locator objects
- ✅ The above methods not to be **"awaited"** as it does not return a promise
- ✅ The type of locator is an "object"
- ✅ Locators are LAZY

Codegen helps find the right locator

Cope

Handling Web Elements

Cope

Element Type: Button/Link

Actions

1. Click
2. Press
3. Double click
4. Right Click
5. Hover
6. Add timeout if slow

Cope

Debugging Tests

Cope

Playwright Debugging Options

Options

1. VS Code Test Explorer
2. UI Mode (--ui)
3. Debug (PWDEBUG=1)
4. Trace Viewer

Cope

Allure Reporter

Cope

🎯 Allure Reporter

Why Allure Reporter

1. Rich UI
2. Frameworks Agnostic - Supports different frameworks like selenium, wdio and playwright
3. Detailed insights into Test Execution
4. Trend Tracking
5. Works in CI/CD

Cope

Playwright Core Capabilities

Cope

Test Config Overview

As a Quality Engineer,

I want to learn more about the Playwright config file

So that I can understand the available configuration options and know when and where to use them.

Cope

Fixtures

As a Quality Engineer,

I want to learn more about the playwright in-built fixtures

So that I can write cleaner and more maintainable test code.

Cope

What is Fixture?

In simple terms, **"fixtures"** refers to **reusable** pieces of **setup or test context** that are required before running tests.

Fixtures solves one of the main problems of **DRY (Don't Repeat Yourself)**

*In other words, it's a **pre-built objects** loaded with actions*

Cope

Annotations

As a Quality Engineer,

I want to add additional information to the tests

So that I can better understand the test or test suite.

Cope

Tags

As a Quality Engineer,

I want to group tests

So that I can run them as a group (e.g., 'smoke', 'regression').

Cope

Playwright Advanced Capabilities

Cope

🤔 Sounds familiar to you?

implicitWait

explicitWait

sleep()

waitForDisplayed

pause()

Cope

Auto-Waiting

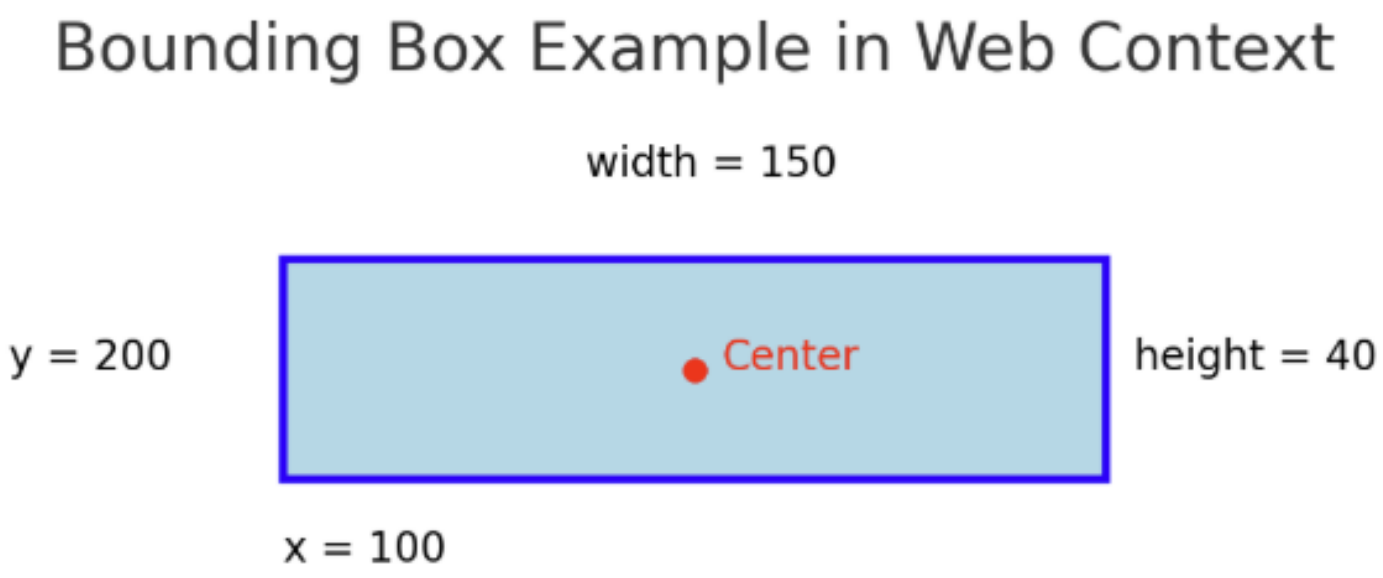
As a Quality Engineer,

I want understand Playwright's auto-waiting feature

So that I implement them in my code so it is less flaky and avoid manual waits

Cope

Bounding Box



⏳ Timeout

As a Quality Engineer,

I want understand Playwright's timeout setting

So that I set and configure the right timeout for my test suite

Order (highest → lowest):
Test-level → CLI → Config timeout

Cope

🎯 Browser Args

As a Quality Engineer,

I want configure additional browser arguments

So that can overcome challenges when running tests in Chrome locally or in the CI environment.

Cope

🎯 Hooks

As a Quality Engineer,

I want understand and use Playwright hooks

So that I can set up preconditions and clean up after my tests run in a structured way.

Cope

Hooks - Summary points

🎯 **Remember**

1. Hook can be at test file level or at `describe` level

2. The most common one being -> `describe -> beforeEach -> test`

Cope

🎯 Global Setup & Teardown

As a Quality Engineer,

I want to define **one-off** tasks such as setup and teardown activities

So that I have an ability to perform setup and teardown process in more controlled manner

Cope

🎯 [Mobile Web] Device Emulation

As a Quality Engineer,

I want execute tests across multiple **devices** and browsers

So that I can ensure the application works consistently across different environments.

Cope

🎯 Understanding Playwright Parallelism

As a Quality Engineer,

I want to run tests in parallel

So that I can reduce execution time and achieve faster feedback.

Cope

🎯 Setting up ‘tsconfig.json’ file

As a Quality Engineer,

I want to maintain a TypeScript configuration file

So that I can customize project settings to align with specific requirements.

Cope

Understanding Data

Cope

What data goes where? - Organizing Data the Right Way

password

Test Data(data parameterization)

App URL

SQL Query Template

System Generated Auth Token

API Endpoint(/user)

1.Static Data (constants.json)

2.Test Data (parameterization)

3.Dynamic Data (Global setup)

4.Env Config (config file)

5.Sensitive Data (.env)

Cope

Data Handling With CSV Files

Cope

🔥 Why CSV Files ?

Benefits of using CSV files

1. Easier to **maintain** and **scale** test data

2. **Non-technical team member** can update test data incl BA's and Business team

3. Better for **data-driven testing** (i.e. Running same scenario with different set of data)

Cope

Framework Building

Cope

Core Characteristics of Test Framework

Core Components

1.Modular

2. Maintainable

3. Extendable

4.Usable

5.Reporting

🎯 Page Object Model

As a Quality Engineer,

I want to set up a well-known Page Object Model (POM) design pattern

So that I can leverage its design principles to build a scalable and maintainable test framework.

Cope

🎯 Page Object Model

What's a page object?

A **page** or section of a web page as **an object**.

In technical terms, an object can have **properties (i.e elements) and methods (i.e. actions)**

Cope

API Testing Integration

Cope

🎯 Why API Testing?

Focus on the Logic, Not the UI

API testing verifies the **back-end logic** of an application by sending **HTTP requests** (GET, POST, PUT, DELETE) directly to endpoints — without using the front-end interface

Cope

🎯 HTTPS Methods

📦 Common HTTP Methods Explained

Method	Purpose	Example
GET	Read/fetch data	`GET /students` → list all students
POST	Create a new item	`POST /students` with body `{name: "John"}`
PUT	Update an existing item	`PUT /students/123` with body `{name: "Johnny"}`
PATCH	Partially update an item	`PATCH /students/123` with `{age: 22}`
DELETE	Remove an item	`DELETE /students/123`

> 💡 All of these are just different **verbs** that describe what action you want to do with data on the server.

Cope

MCP Server Overview

Cope

🎯 What is MCP?

It is **Model Context Protocol (MCP)**

MCP is an open protocol that standardizes how applications provide context to LLMs.

Cope

GitHub Copilot - Modes

Cope

🎯 How many Chat Modes in VS Code?

🎯 4 Modes

1. Inline Mode
2. Ask Mode
3. Edit Mode
4. Agent Mode

Simple Tasks

↓

Complex Tasks

Cope

🎯 Inline Mode

Definition:
Ask copilot in the **opened editor** or terminal without opening the chat window.

Tip: Use `Ctrl + I` or `Command + I` (macOS)

Examples:
"Explain me this variable"
"Add a variable to this class"

Cope

🎯 Ask Mode

Definition:
Optimized for asking questions about your **codebase, coding, and general technology concepts**

Tips: Generally, explain/how/what/where questions

Examples:
"Explain me map function used in this file"
"How to add a new test to this spec file Give an example"

Cope

🎯 Edit Mode

Definition:
Optimized for making **code edits** across multiple files in your project. VS Code directly applies the code changes in the editor, where you can review them in-place

Tips: Add/Refactor/update something at file level for better outcome

Examples:
"Based on given selectors, add a page objects"
"Refer to the login flow and create another test for negative flow"

Cope

🎯 Agent Mode

Definition:
Optimized for coding tasks based on high-level requirements and handling **complex tasks** which involved external tools and terminal commands

Tips: Tasks at more of project levels

Examples:
"This is my URL, capture the login flow and create a spec file"
"Refactor the codebase so I can use cucumber BDD"

Cope

🎯 Remember this 3s

3s Principle:

1. Simple
2. Specific
3. Short

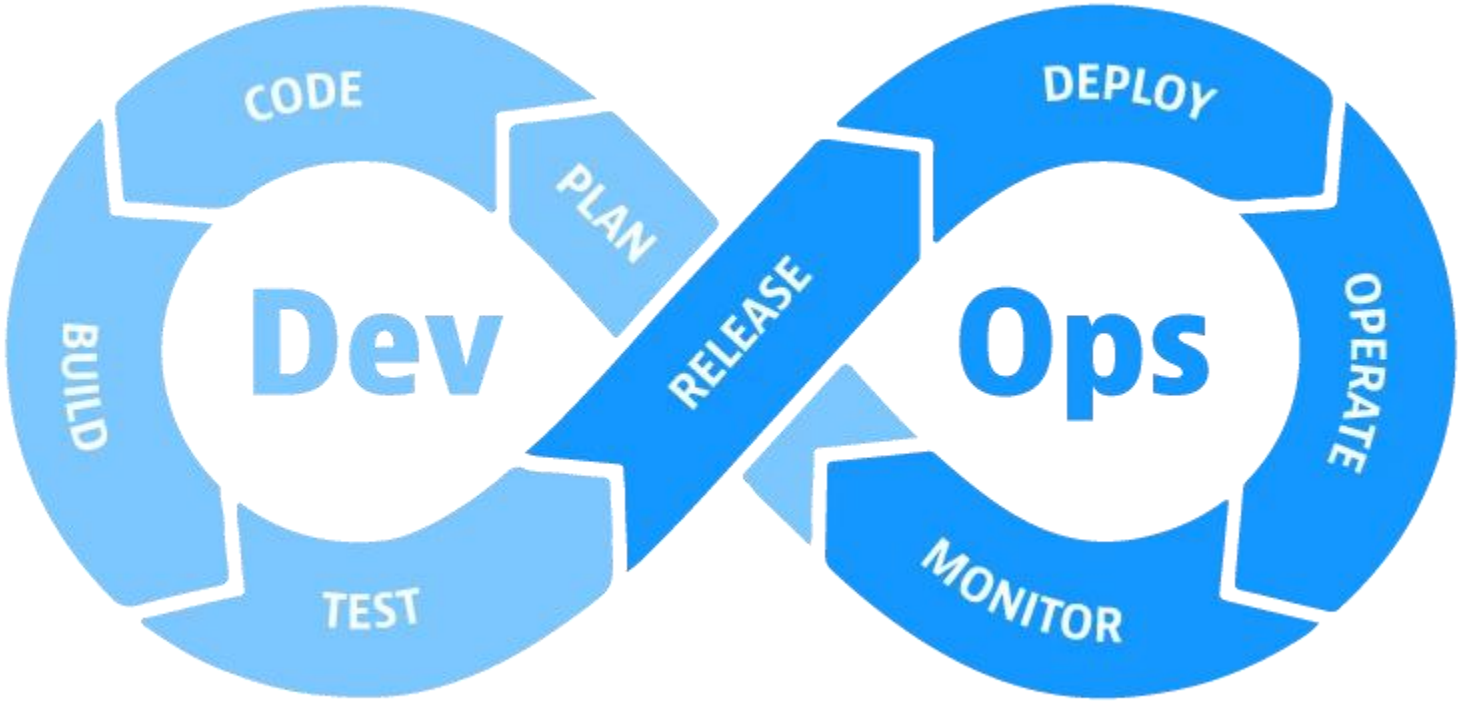
Cope

DevOps & CI - Overview

Cope

🎯 DevOps - The Big Picture

DevOps is a concept or *set of practices* that bridges the gap between **development** and **operations**.



Cope

🎯 DevOps - Key Practices

Key Practices

1. CI - Continuous Integration
2. CD - Continuous Delivery / Deployment
3. Infrastructure as Code (IaC)
4. Monitoring & Logging
5. Automation & Configuration Management

Cope

🎯 Continuous Integration

CI focuses on *automating the **build** and **test** process* every time a developer commits code.

CI Tools

1. GitHub Actions
2. Jenkins
3. Azure Pipelines
4. CircleCI
5. GitLab CI

Cope

Overview of YML/YAML file

Cope

🎯 YML/YAML - Fast Facts

Key Practices

1. CI - Continuous Integration
2. CD - Continuous Delivery / Deployment
3. Infrastructure as Code (IaC)
4. Monitoring & Logging
5. Automation & Configuration Management

Cope

Jenkins Integration

Cope

🎯 What is Jenkins?

A "**butler**" (chief manservant of a house), who does all the work for you automatically

So, **Jenkins = your automation butler**, who builds, tests, and deploys software tirelessly for you

Cope

🎯 What is Jenkins?

The leading open source automation server, Jenkins provides hundreds of plugins to support building, deploying and automating any project.

- Jenkins is a **open source**
- Jenkins runs on **Java**

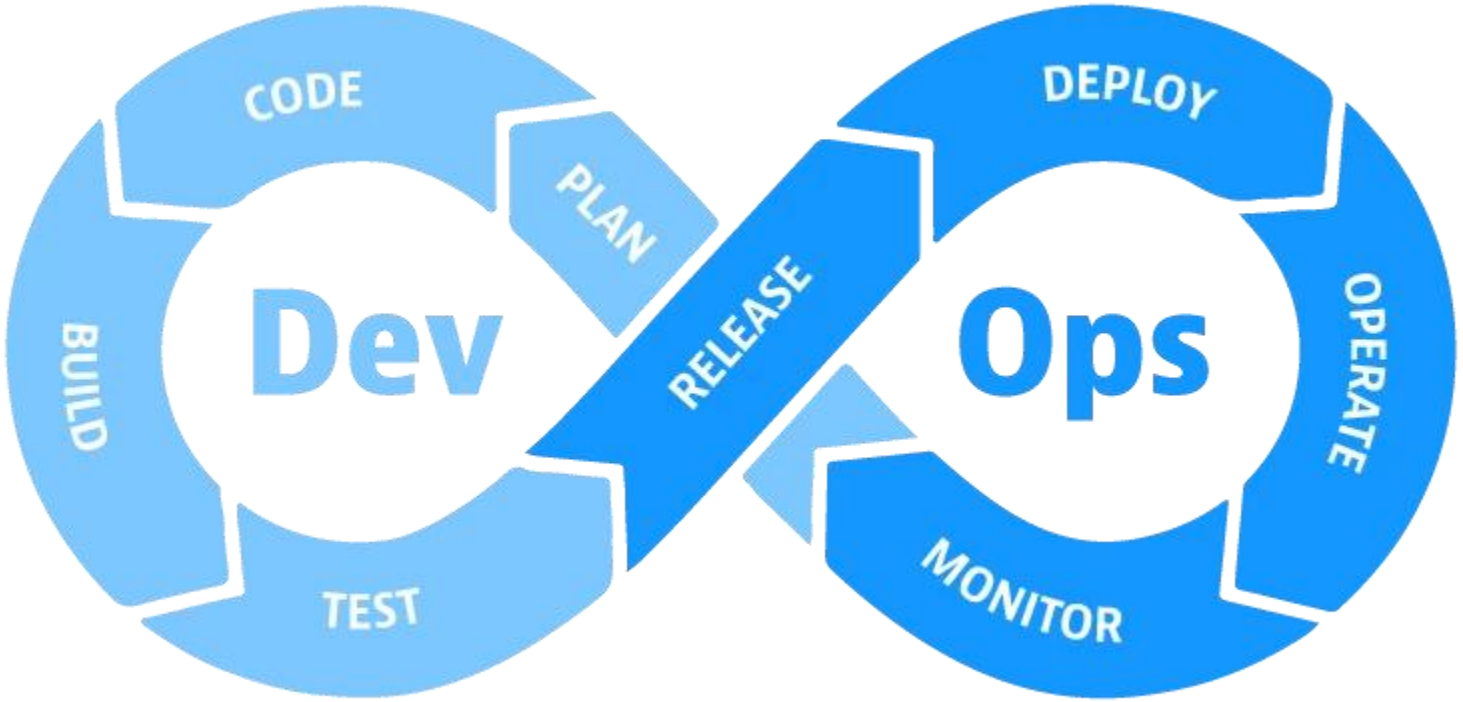
Cope

Jenkins Pipeline Setup

Cope

🎯 [Recap] - DevOps - The Big Picture

DevOps is a concept or *set of practices* that bridges the gap between **development** and **operations**.



Cope

What's a pipeline?

What we see here:

1. Stages
2. A set of tools and equipments that forms a complete "pipeline"

Running a Jenkins pipeline

Cope

🔥 Quick Facts About Jenkins file

Quick 5 Facts

1. The standard name is "**Jenkinsfile**"
2. It's written in `**Groovy**`
3. Generally it will have few stags such as "**Build**" and "**Test**" with a post action
4. Can configure "**any**" agents or "**docker**" images
5. Sits in the `**root**` dir

Cope

Playwright Test Agents

Cope

🎯 What We Know So Far?

Things we know already…

1.Codegen - A smart test flow recorder

2. MCP Server

3. Copilot - VS Code in-built integration

Why do we need AI Agents?

Cope

🎯 Playwright Test Agents - A Quick Overview

[Q]: How do we describe these agents in one liner?

In a way, these agents can be described as ***"Advanced MCP"***

[Q]: How many agents ?

Three. **1. Planner; 2. Generator; 3. Healer**

[Q]: How to use them?

Can be used independently, or sequentially. Using **sequentially** will produce a good test coverage

Cope

🎯 Playwright Agents - Definition

Planner	Explores the Application *In: a seed-test/PRD*	• A test plan as (.md) file
Generator	Converts the .md file to executable pw tests *In: test plan(md file)*	• A spec file (*.spec.ts)
Healer	Fixes any failing test(s) *In: A spec file/failing test*	• A error-free spec file

Cope

Playwright Test Agents - Key Takeaways

What works well?

- An extensive **test plan** based on PRD or application under test
- A ready-to-use **boilerplate code** with zero effort
- An objective driven agent can be used in isolation (e.g. Healer agent)
- A productivity boost

Cope

🔥 Playwright Test Agents - Key Takeaways

What to be considered?

1.Feels overwhelming and requires manual review

2. Assertions are not just always visibility checks - needs business rule checks

3. Framework compliant - The generated code serves the base purpose

4. Security concerns - exposing the creds and API keys needs review

Cope

Forking an Existing Playwright Repository for Faster Setup

Cope

🎯 Clone Vs Fork Use Cases

Clone Use case

👤 You found a GitHub repo and want to ***look at the code*** *or run it locally without owning it*.

Fork Use case

👤 You found a GitHub repo and want to ***own editable copy of the code*** and build on top of it.

Cope

🍴 ”Fork” – FAQs

[Q]: What are the benefits of “Forking” a repo?

You can **own** a personal copy of the repo; Can **customize** it for your own need; Stay up-to-date with upstream.

[Q]: After I fork a repo, is that “Read-only” or “Editable” ?

It’s **editable** and can make your own changes and keep it under your account

[Q]: When I fork a repo, will the upstream owner know that I forked?

Yes and No. Yes- the owner knows someone forked butha

[Q]: Can I delete the repo once I forked?

Yes, you can as it is **your own copy.** The upstream owner will not know about it.

Cope

🍴 ”Fork” – FAQs

[Q]: What are the commands used in Fork?

```
# Clone
git clone https://github.com/<your-username>/<forked-repo>.git

# Track upstream
git remote add upstream https://github.com/<original-author>/<repo>.git

# Check remote
git remote -v

# Pull latest from upstream
git pull upstream main

# You can push your changes - this does not impact the upstream repo
git push origin main
```